

Collision Attacks on SHA-256 up to 37 Steps with Improved Trail Search

Zhuolong Zhang^{1,3,4}, Muzhou Li^{2,3,4,5}, Lei Gao^{1,3}, and Meiqin Wang^{1,3,4}(✉)

¹ School of Cyber Science and Technology, Shandong University, Qingdao, China
zhuolongzhang@mail.sdu.edu.cn, {leigao, mqwang}@sdu.edu.cn

² School of Cryptologic Science and Engineering, Shandong University, Jinan, China
muzhouli@mail.sdu.edu.cn

³ State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China

⁴ Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Jinan, China

⁵ Quan Cheng Laboratory, Jinan, China

Abstract. As a NIST-standardized hash function, SHA-256 has been extensively analyzed in the context of collision attacks. Although Mendel et al. introduced the first 31-step collision attack at EUROCRYPT 2013, the number of attacked steps has not been increased for more than a decade. After a thorough review of existing attacks, we identify that progressing beyond 31 steps necessitates new local collisions in the message expansion. To date, such local collisions have been manually constructed, which is both time-consuming and technically challenging. Besides, even the latest automated models for searching signed differential trails fail to accurately account for the bit conditions imposed by Boolean functions, potentially overlooking high-quality trails. In this paper, we enhance the existing framework by overcoming these two limitations. Firstly, an automated tool that can efficiently identify high-quality local collisions is given following the main idea of EUROCRYPT 2013. Secondly, two new models of Boolean functions that can accurately capture bit conditions are introduced. Leveraging these improvements, we finally reach the first 37-step collision attack on SHA-256, extending the number of attacked steps by six, and this is the first such advancement in 12 years.

Keywords: Collision Attack · Local Collision · SHA-256 · SAT/SMT

1 Introduction

The MD-SHA family of hash functions has long been a cornerstone of modern cryptography, providing essential primitives for message integrity, digital signatures, and secure protocols. However, since 2005, Wang et al. [21] introduced techniques such as signed differences and message modifications, which had a devastating impact on the MD-SHA family: several widely used hash functions were subsequently broken, including MD4 [21], RIPEMD [21], MD5 [23], SHA-0 [1, 24], SHA-1 [5, 6, 20, 22], and RIPEMD-128 [4]. In contrast, RIPEMD-160

and the SHA-2 series have survived these attacks and still maintain a substantial security margin against collision attacks. Among these, SHA-2 is of particular practical and theoretical importance and is widely employed in security-critical applications worldwide, including Bitcoin. Therefore, this work focuses on investigating collision attacks on SHA-2, aiming to deepen the understanding of its structural security.

In the study of collision attacks on SHA-2, considerable efforts have focused on advancing automated search techniques. At ASIACRYPT 2011, Mendel et al. [13] introduced the first automated tool based on the guess-and-determine (GnD) approach for identifying complex differential characteristics in SHA-256. This tool is further advanced by the same team at EUROCRYPT 2013 [14]. It can discover distinguishers covering more steps by reducing the search space and employing a three-stage search strategy instead of the earlier two-stage process. For SHA-512, the larger word size significantly increased the search space, preventing successful attacks. Eichlseder et al. [2] addressed this at FSE 2014 by improving the GnD branching heuristic to discover differential characteristics and conforming message pairs. However, all the above tools were not released publicly. To address the absence of open-source tools and better capture signed differential propagation, Li et al. re-implemented the model built for RIPEMD-160 shown in [12] with a SAT/SMT-based tool and applied it to SHA-2 at EUROCRYPT 2024 [8] and ASIACRYPT 2024 [9]. The primary objective of these tools has been to minimize the Hamming weight of the differential characteristic. However, the full probability also depends on the number of bit conditions imposed by Boolean functions, which was ignored. Most recently, at CRYPTO 2025, Li et al. [10] further enhanced automated search by introducing a new modeling technique that incorporates "2-bit conditions" in signed differential transitions through Boolean functions. Nonetheless, this approach still remains incomplete, as it considers only a subset of the bit conditions. Consequently, as shown in [10], multiple attempts are still required during the search procedure to determine the minimal number of differential conditions. This indicates that existing models do not fully capture all conditions. Besides, there exist some non-necessary conditions that are included in the objective function, which also force one to check its actual probability manually after obtaining the differential trail.

Additionally, regardless of whether GnD-based tools or off-the-shelf solvers are used, the critical step in collision attacks on SHA-2 is identifying a relatively complex local collision for the message expansion phase, as this determines the number of attacked steps. Several major advances have been achieved through the construction of new local collisions. At ASIACRYPT 2011, Mendel et al. [13] presented the first 27-step local collision on SHA-256 by exploiting the nonlinearity of both the state update and message expansion, thereby extending the best known attacks from 24 to 27 steps. Subsequently, Mendel, Nad and Schl  ffer [14] introduced several principles of choosing local collisions. Benefiting from these principles, they proposed the first 28-step and 31-step collisions on SHA-256. More recently, Li et al. [8] reported the first 28-step practical and 31-step theoretical collision attacks on SHA-512, following the same local collision strategy

as in [14]. At CRYPTO 2025, Li et al. [10] analyzed the message expansion in detail and introduced a new 29-step local collision, thereby improving the practical attacks on SHA-512 by one step. After reviewing all the above proposed attacks, one can realize that a better choice of local collision is essential to mount attacks covering more steps. However, these results were either derived from manually constructed local collisions or obtained by transferring local collisions used in attacking SHA-256 to SHA-512 due to their structural similarity. In the latter case, they essentially rely on the same local collision. Given the time-consuming and highly technical nature of manual construction, and the fact that the number of attacked steps has stagnated at 31 for more than a decade, a pressing question is whether new and better local collisions can be discovered to advance the state of the art.

1.1 Contributions

To address the reliance on manually constructed local collisions and the limitations of existing automated search models for SHA-2, our contributions can be summarized as follows:

New Local Collisions. To overcome the limitations of manual constructions, we developed an automated tool to search for high-quality local collisions in SHA-2 (detailed in Section 3). The tool describes difference transitions through the message expansion at the word level and uses both the number of message words with non-zero differences and cancellation conditions as the objective function. Using this tool, we identify a 37-step local collision ($W_6, W_7, W_9, W_{14}, W_{22}, W_{23}$) and a 36-step local collision ($W_5, W_7, W_8, W_{13}, W_{21}, W_{22}$). Both local collisions exhibit advantages over the previous 31-step one regarding the number of message words with non-zero differences and cancellation conditions, leading to our 37-step and 36-step collision attacks.

More Holistic Model. In SHA-2, the probability of a differential characteristic depends on two factors: the Hamming weight and the number of bit conditions derived by Boolean functions. However, existing models either ignore the latter factor or capture only part of the conditions. To address this limitation, we introduce two new models for Boolean functions in Section 4. The first explicitly accounts for all bit conditions in the IF function, while the second models IF and MAJ with variable-specific conditions to more precisely capture constraints on individual inputs. These models enable more effective control over the uncontrolled conditions, thereby improving the accuracy of probability estimation and the success rate of collision attacks. We implement the new automatic search tool using the SAT/SMT solver STP¹.

Applications to SHA-256. Combining the local collisions discovered in Section 3 with our more holistic automatic search tool, we present the first collision

¹ <https://stp.github.io/>

attack on 37-step SHA-256 with time complexity $2^{119.1}$, improving the previous best result by 6 steps after more than a decade. In addition, we also provide the first 36-step collision attack with time complexity $2^{94.4}$. For comparison, representative collision and semi-free-start (SFS) collision attacks are summarized in Table 1.

Table 1: Summary of (semi-free-start) collision attacks on SHA-256.

Attack type	Attack Steps	Time	Memory	References	Year
collision	24	practical	\	[3, 18]	2008
	27	practical	\	[13]	2011
	28	practical	\	[14]	2013
	31 [†]	$2^{65.5}$	2^{34}	[14]	2013
		$2^{49.8}$	2^{48}	[8]	2023
		<i>practical</i>	$2^{19.8}$	[9]	2024
	37[†]	$2^{122.2}$	$2^{34.9}$	Sect. 5.1	2025
		$2^{119.1}$	$2^{39.9}$		
SFS collision	36[†]	$2^{94.4}$	$2^{12.6}$	Sect. 5.2	2025
	38	practical	\	[14]	2013
	39	practical	\	[8]	2023

[†] It is a two-block collision attack.

To allow our results to be reproduced, all codes are available at the anonymous website:

<https://github.com/Zhang-SDU/AutoSHA2Collision>.

1.2 Organization

The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries, including notations and a brief overview of SHA-2. In Section 3, we present our automated search tool for local collisions and describe the new 37-step and 36-step local collisions it identifies. Section 4 introduces our improved modeling of Boolean functions, including the full conditions and variable-specific models. In Section 5, we combine these improvements to mount collision attacks on 37-step and 36-step SHA-256. Finally, we conclude the paper in Section 6.

2 Preliminaries

2.1 Notations

Denote $x[i]$ as the i -th bit of x , where $x[0]$ is the rightmost bit. The signed difference between x' and x is denoted by ∇x . Specifically, $\nabla x[i] = [\mathbf{n}]$ if $x[i] = 0$ and $x'[i] = 1$; $\nabla x[i] = [\mathbf{u}]$ if $x[i] = 1$ and $x'[i] = 0$; $\nabla x[i] = [=]$ if $x'[i] = x[i]$; $\nabla x[i] = [0]$ if $x'[i] = x[i] = 0$; $\nabla x[i] = [1]$ if $x'[i] = x[i] = 1$.

The Hamming weight of ∇x , denoted by $\mathbb{H}(\nabla x)$, is defined as the total number of $\mathbf{n}$ and $\mathbf{u}$ in ∇x . In addition, $\boxplus$ and $\boxminus$ represent the modular addition and subtraction on 32/64 bits, respectively. The modular difference is denoted by δx , i.e. $\delta x = x' \boxminus x$. Symbols $\gg$, $\ggg$, $\oplus$ and $\wedge$ represent shift right, rotate right, exclusive or, and, respectively.

2.2 Description of SHA-2

The SHA-2 family, based on the Merkle-Damgård structure, is specified by NIST as part of the Secure Hash Standard (SHS) [16]. The standard defines two primary versions: SHA-256 and SHA-512, which use 32-bit and 64-bit state words, respectively. For convenience, we denote the state size as n bits, and thus $n \in \{32, 64\}$. Each compression function takes a $16n$ -bit message block and an $8n$ -bit chaining variable as inputs, generating an $8n$ -bit output. The compression functions of SHA-2 are computed through iterative updates of the internal state. Each update is called a step. The number of steps r is 64 for SHA-256 and 80 for SHA-512.

In the following, we provide a brief overview of the compression functions, which consist of three main components: message expansion, state update transformation, and finalization. For more information, one can refer to [16].

Message Expansion. The $16n$ -bit message block is divided into 16 message words $(m_0, m_1, \dots, m_{15})$, each of size n bits. Then, these words are expanded to r expanded message words W_i as follows:

$$W_i = \begin{cases} m_i, & 0 \leq i \leq 15, \\ \sigma_1(W_{i-2}) \boxplus W_{i-7} \boxplus \sigma_0(W_{i-15}) \boxplus W_{i-16}, & 16 \leq i \leq r-1. \end{cases} \quad (1)$$

In SHA-256, the functions $\sigma_1(x)$ and $\sigma_0(x)$ are defined as

$$\begin{aligned} \sigma_1(x) &= (x \ggg 17) \oplus (x \ggg 19) \oplus (x \gg 10), \\ \sigma_0(x) &= (x \ggg 7) \oplus (x \ggg 18) \oplus (x \gg 3). \end{aligned}$$

In SHA-512, the functions $\sigma_1(x)$ and $\sigma_0(x)$ are defined as

$$\begin{aligned} \sigma_1(x) &= (x \ggg 19) \oplus (x \ggg 61) \oplus (x \gg 6), \\ \sigma_0(x) &= (x \ggg 1) \oplus (x \ggg 8) \oplus (x \gg 7). \end{aligned}$$

State Update Transformation. Figure 1 illustrates the state update of SHA-2, which updates an $8n$ -bit chaining variable $IV = (A_{-1}, \dots, A_{-4}, E_{-1}, \dots, E_{-4})$ in the i -step as follows:

$$\begin{aligned} E_i &= A_{i-4} \boxplus E_{i-4} \boxplus \Sigma_1(E_{i-1}) \boxplus \text{IF}(E_{i-1}, E_{i-2}, E_{i-3}) \boxplus K_i \boxplus W_i, \\ A_i &= E_i \boxplus A_{i-4} \boxplus \Sigma_0(A_{i-1}) \boxplus \text{MAJ}(A_{i-1}, A_{i-2}, A_{i-3}), \end{aligned}$$

where $0 \leq i \leq r-1$ and K_i is a predefined constant in the i -th step. Here, same Boolean functions IF and MAJ are utilized in SHA-256 and SHA-512, which are defined as:

$$\begin{aligned} \text{IF}(x, y, z) &= (x \wedge y) \oplus (x \wedge z) \oplus z, \\ \text{MAJ}(x, y, z) &= (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z). \end{aligned}$$

However, as for the linear functions Σ_1 and Σ_0 , they are different. Specifically,

$$\begin{aligned} \Sigma_1(x) &= (x \ggg 6) \oplus (x \ggg 11) \oplus (x \ggg 25), \\ \Sigma_0(x) &= (x \ggg 2) \oplus (x \ggg 13) \oplus (x \ggg 22) \end{aligned}$$

are used in SHA-256, while

$$\begin{aligned} \Sigma_1(x) &= (x \ggg 14) \oplus (x \ggg 18) \oplus (x \ggg 41), \\ \Sigma_0(x) &= (x \ggg 28) \oplus (x \ggg 34) \oplus (x \ggg 39). \end{aligned}$$

are used in SHA-512.

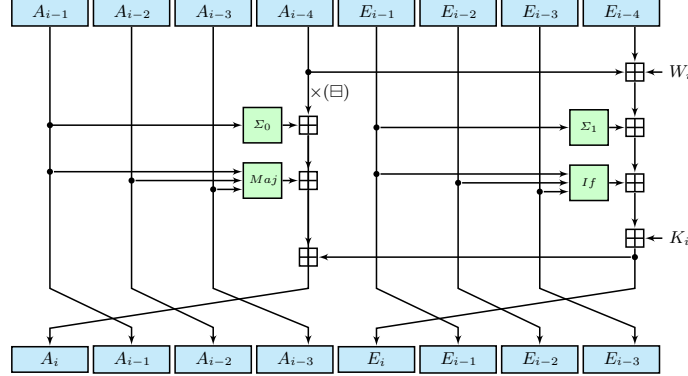


Fig. 1: The state update transformation of SHA-2.

Finalization. The compression functions of SHA-2 employ the Davies-Meyer construction. After performing the step update transformation for r times, the previous chaining value is added to the output of the state update. The result of this feed-forward sum yields the new chaining value h :

$$h = (A_{r-1} \boxplus A_{-1}, \dots, A_{r-4} \boxplus A_{-4}, E_{r-1} \boxplus E_{-1}, \dots, E_{r-4} \boxplus E_{-4}).$$

Our Attacks Setting. Assuming $f(CV, M)$ denotes the compression function, our collision attacks on SHA-256 target two message blocks $M_0 || M_1$ and $M_0 || M'_1$ such that

$$CV_1 = f(CV_0, M_0), \quad f(CV_1, M_1) = f(CV_1, M'_1),$$

where $M_1 \neq M'_1$ and CV_0 is the fixed initial value defined in the specification of SHA-2. An illustration of this attack setting is shown in Figure 2. Notice that if we start from the second block, it can be regarded as a semi-free-start collision attack. Unlike collision attacks, the chaining value can be freely chosen in this case. By adding another block ahead, one can convert the SFS collision attack into a collision one, which has already been revealed by [14].

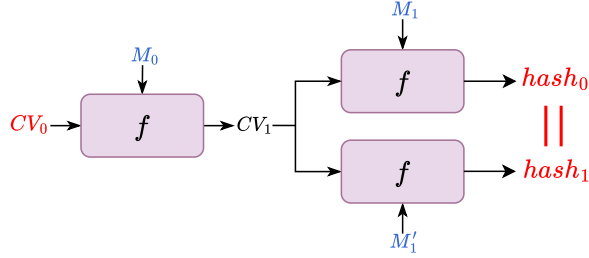


Fig. 2: Two-block collision attack framework for SHA-256.

3 Search for New Local Collisions of Message Expansion

In collision attacks on SHA-2, the most critical step is first to identify a relatively complex local collision in the message expansion, as it determines the number of steps that can be attacked. Since the first 31-step collision attack on SHA-256 was proposed in 2013, no further progress has been made on covering more steps. Reviewing previous work on SHA-2, we found that one possible way to extend attacks to cover more steps is to discover new local collisions. However, relatively little work has focused on this aspect. In this section, we provide an automated tool to search for local collisions following the way shown by Mendel et al. [14]. We first recall a formal definition of local collision.

Definition 1. A *local collision* refers to a set of message words that contain differences. Within this set, the first message word exhibiting a difference is defined as the **start step**, while the last such word is termed the **end step**. The **span** denotes the number of steps from the **start step** to the **end step**.

Example 1. If a local collision is $(W_{i_1}, W_{i_2}, \dots, W_{i_j})$ where $i_1 < i_2 < \dots < i_j$, then i_1 is the **start step** and i_j is the **end step**. The **span** is given by $i_j - i_1 + 1$.

In previous works such as [13, 17–19], most of them relied primarily on manually constructing local collisions, by carefully exploiting the nonlinearities of both the state update and the message expansion. In fact, it is also possible to focus solely on the message expansion, as demonstrated by Li et al. at CRYPTO 2025 [10], where they performed a detailed analysis of the message word expansion and proposed a new 29-step local collision. Nevertheless, such hand-crafted constructions remain highly technical and extremely time-consuming.

Therefore, a tool that can automatically search for local collisions is both necessary and meaningful. Interestingly, as early as EUROCRYPT 2013, Mendel et al. [14] had already presented an idea for constructing local collisions that is relatively easy to understand. However, this insight appears to have gone largely unnoticed by the research community for more than a decade.

Here, we first review the main ideas of [14] and then implement a simple SAT/SMT-based tool following its main idea. Finally, we identify many new local collisions. The best of them enables us to achieve the first collision attack on 37-step SHA-256, which improves the previous best collision attack on SHA-256 by 6 steps.

Main Idea of Constructing Local Collisions Proposed in [14]. According to the message expansion of SHA-2, when $i \geq 16$,

$$W_i = \sigma_1(W_{i-2}) \boxplus W_{i-7} \boxplus \sigma_0(W_{i-15}) \boxplus W_{i-16}.$$

The method in [14] does not consider the exact differential propagation through the functions σ_0 and σ_1 , but instead employs a word-level property that considers only whether a message word has a non-zero difference. Specifically,

1. If all four input words ($W_{i-2}, W_{i-7}, W_{i-15}, W_{i-16}$) have no differences, then W_i necessarily has no difference.
2. If exactly one of the four input words has a non-zero difference, then W_i necessarily has a non-zero difference.
3. If at least two input words have non-zero differences, their contributions may either cancel or reinforce each other. In this case, differences of W_i can be zero or not.

Since a lot of degrees of freedom provided by the message words are required to satisfy all the conditions imposed by the differential characteristic, Mendel et al. pointed out the importance of using as few message words with non-zero differences as possible [14]. Moreover, to achieve a large number of steps without injecting differences, they required that differences of the expanded message words cancel each other out. The cancellation of differences introduces additional constraints on the message words involved in the expansion. Consequently, minimizing the number of message words with differences reduces the number of such conditions, and ensuring that the cancellation conditions themselves remain few is equally essential.

SAT/SMT-based Tool to Search for Local Collisions. Based on the main idea described above, we implement a simple tool to automatically find good local collisions that cover more steps while requiring fewer constraints. To construct the model, we first introduce a binary variable ΔW_i to represent whether the message word has a non-zero difference. Then we define a 1-bit auxiliary variable $flag_i$ to capture the cancellation conditions when computing differences on W_i for $i \geq 16$. For the message expansion, we constructed a table of all 27 valid transitions of the form $(\Delta W_{i-2}, \Delta W_{i-7}, \Delta W_{i-15}, \Delta W_{i-16}) \rightarrow (\Delta W_i, flag_i)$, as shown in Table 2.

Table 2: Valid values of $(\Delta W_{i-2}, \Delta W_{i-7}, \Delta W_{i-15}, \Delta W_{i-16}) \rightarrow (\Delta W_i, flag_i)$.

[0000 $\rightarrow$ 00],
[0001 $\rightarrow$ 10], [0010 $\rightarrow$ 10], [0100 $\rightarrow$ 10], [1000 $\rightarrow$ 10],
[0011 $\rightarrow$ 01], [0011 $\rightarrow$ 10], [0101 $\rightarrow$ 01], [0101 $\rightarrow$ 10],
[1001 $\rightarrow$ 01], [1001 $\rightarrow$ 10], [0110 $\rightarrow$ 01], [0110 $\rightarrow$ 10],
[1010 $\rightarrow$ 01], [1010 $\rightarrow$ 10], [1100 $\rightarrow$ 01], [1100 $\rightarrow$ 10],
[0111 $\rightarrow$ 01], [0111 $\rightarrow$ 10], [1011 $\rightarrow$ 01], [1011 $\rightarrow$ 10],
[1101 $\rightarrow$ 01], [1101 $\rightarrow$ 10], [1110 $\rightarrow$ 01], [1110 $\rightarrow$ 10],
[1111 $\rightarrow$ 01], [1111 $\rightarrow$ 10].

Using the software LogicFriday², we can generate the minimized conjunctive normal form (CNF) formula encoding Table 2, denoted as C_W . This formula ensures that any assignment to the message difference variables (ΔW_i) and the auxiliary variables $(flag_i)$ is consistent with one of the valid transitions listed in Table 2.

By embedding C_W into the SAT/SMT solver, we guarantee that only feasible difference patterns are explored during the search. To complete the entire search model, four additional parameters (R, V, K, C) are introduced to specify the initial constraints and the objective function. The overall procedure for finding a local collision is summarized in Algorithm 1. To identify good local collisions, we can gradually decrease the bound C until the solver outputs no solution.

Candidates of Good Local Collisions for SHA-2. The best known collision attack targets 31 steps of SHA 256 [8, 9, 14] and SHA-512 [8–10], each relying on a local collision $(W_5, W_6, W_7, W_8, W_9, W_{16}, W_{18})$ with a span of $t = 14$ steps. To launch a collision attack covering more steps, it is necessary to increase the span t . Specifically, we examine all cases with $14 \leq t \leq 25$. Furthermore, to turn a semi-free-start collision into a collision attack, Mendel et al. [14] pointed out that it is necessary to impose no differences in the first (ideally 8) message words. Enabled by the efficiency of our tool, we exhaustively explore all cases with *start step* $V \geq 0$. Using this strategy, we obtain many candidate local collisions where the attack step $R \geq 32$. The last message word W_i of a local collision can be

² <https://download.cnet.com/>

used to achieve zero difference in the last 4 words $(E_{i-3}, \dots, E_i)$ and the last 8 words $(A_{i-7}, \dots, A_i)$. This is necessary to produce zero differences in the final 4 steps of the state update and lead to a collision. Besides, since conditions after (A_{16}, E_{16}) are always hard to satisfy, the number of such conditions must be kept relatively small. In this case, $V + K$ cannot be too large. Following these considerations, we obtain two good local collisions³, shown in Table 3.

Algorithm 1 Searching for a local collision with fixed attack step R , start step V , span K , and objective bound C

```

procedure Search_Local_Collision( $R, V, K, C$ )
  if  $V + K > R$  then // invalid parameters
    return "invalid"
  Declare variables  $\Delta W_0, \Delta W_1, \dots, \Delta W_{R-1}$  of size 1
  Declare variables  $\Delta flag_{16}, \Delta flag_{17}, \dots, \Delta flag_{R-1}$  of size 1
  Declare a vector  $OBJ$  of size 7 // objective value to minimize
  Initial_Set( $R, V, K$ )
  for  $i = 16$  to  $R$  do
    add constraints  $C_W(\Delta W_{i-2}, \Delta W_{i-7}, \Delta W_{i-15}, \Delta W_{i-16}, \Delta W_i, t_i)$ 
  Objective_Set( $R, V, K, C, OBJ$ )

procedure Initial_Set( $R, V, K$ ) // initialize constraints
  add constraint  $\Delta W_V = 1$  // start step
  add constraint  $\Delta W_{V+K-1} = 1$  // end step
  for  $i = 0$  to  $R$  do
    if  $i < V$  or  $i > V + K - 1$  then
      add constraint  $\Delta W_i = 0$ 

procedure Objective_Set( $R, V, K, C, OBJ$ ) // set objective function
   $OBJ = 0$ 
  for  $i = V$  to  $V + K$  do
     $OBJ = OBJ + \Delta W_i$ 
  for  $i = 16$  to  $R$  do
     $OBJ = OBJ + flag_i$ 
  add constraint  $OBJ \leq C$ 

```

The first local collision in Table 3 spans over $t = 18$ steps (step 6-23) and results in a collision on 37 steps of SHA-256. The local collision has differences in only 6 message words $(W_6, W_7, W_9, W_{14}, W_{22}, W_{23})$ and we get 6 cancellation conditions at expansion steps 16, 21, 24, 25, 29 and 30. The first 6 message words do not contain any differences and therefore, we can turn a semi-free-start collision into a collision attack below the birthday bound for SHA-256.

³ For 37 steps, we also found another local collision activating the message words $(W_6, W_8, W_9, W_{14}, W_{22}, W_{23})$ and identified a corresponding differential characteristic. However, the resulting attack complexity is too high to yield a valid attack.

Table 3: Message word differences and cancellation conditions for the collision attacks on 37 steps (left) and 36 steps (right). Message words containing differences are shown in red. The symbol "x" indicates the occurrence of a message word in the message expansion equation. For each highlighted row containing an "x", the corresponding message word differences must be canceled, i.e. cancellation conditions.

W_i	6	7	9	14	22	23
0						
1						
2						
3						
4						
5						
6	x					
7		x				
8						
9			x			
10						
11						
12						
13						
14				x		
15						
16			x	x		
17						
18						
19						
20						
21	x			x		
22	x	x				
23		x				
24			x		x	
25			x			x
26						
27						
28						
29				x	x	
30				x		x
31						
32						
33						
34						
35						
36						

W_i	5	7	8	13	21	22
0						
1						
2						
3						
4						
5	x					
6						
7		x				
8			x			
9						
10						
11						
12						
13				x		
14						
15						
16						
17						
18						
19						
20	x			x		
21	x					
22		x				
23		x	x		x	
24			x			x
25						
26						
27						
28				x	x	
29				x		x
30						
31						
32						
33						
34						
35						

Although the above 37-step local collision can be applied to 36 steps, the overall attack complexity remains high for the 36-step collision attack. To address this, we identify another local collision spanning $t = 18$ steps (steps 5–22), with differences in 6 message words ($W_5, W_7, W_8, W_{13}, W_{21}, W_{22}$). This construction enables a collision attack on 36 steps of SHA-256. The key distinction is that it requires only 5 cancellation conditions, occurring at expansion steps 20, 23, 24, 28, and 29 (see Table 3).

4 More Holistic Model with Complete Bit Conditions

In this section, we begin by presenting the motivations for developing a new tool by reviewing the development of modeling techniques for Boolean functions in SHA-2. We then detail our modeling strategies for Boolean functions.

Motivations. The probability of a signed differential characteristic for SHA-2 consists of the following two components:

1. The number of differential bits, i.e., the Hamming weight.
2. The number of bit conditions derived by Boolean functions.

While existing models adequately capture the Hamming-weight aspect, the handling of bit conditions derived from Boolean functions remains incomplete. To this end, when modeling Boolean functions, Li et al. [8, 9], as well as Liu et al. [12], proposed two strategies. The first, referred to as the *fast filtering model*, captures only valid signed difference transitions. The second, known as the *full model*, extends the *fast filtering model* by introducing three monitoring variables to incorporate implicit conditions. However, neither the *fast filtering model* nor the *full model* accounts for the number of these implicit conditions, thus these conditions are not included in the objective function. At CRYPTO 2025, Li et al. [10] observed that an excessive number of "2-bit conditions" arise in A_i , E_i , and W_i which influence the final attack complexity a lot. Thus, they introduced a new modeling approach for Boolean functions by taking the number of these "2-bit conditions" into account. A formal definition of such conditions is described as follows.

Definition 2. [10] *A condition controlling difference propagation is called a "2-bit condition" if it takes the form of either $a = b$ or $a \neq b$, where $a, b \in \mathbb{F}_2$. Here, "2-bit" refers to the fact that the condition involves two bits, but the number of independent constraints imposed by a "2-bit condition" is **only 1**.*

Apart from these "2-bit conditions", there are other conditions derived by the Boolean function IF that should be counted. At ASIACRYPT 2025, Zhang et al. introduced how to count all conditions for several Boolean functions, including IF [25]. Hence, one can reuse the same model to count all conditions for IF. A minor distinction is that they were working on the *fast filtering model*, which is almost the same as the one in the *full model*. For completeness, we reconstruct such a model for IF within the *full model* in Section 4.1.

However, the above method still remains inadequate for optimizing the total number of differential conditions. Both $\text{IF}(x, y, z)$ and $\text{MAJ}(x, y, z)$ are ternary Boolean functions, yet during differential trail search a more accurate counting of bit conditions can often be achieved by examining only one or two of the input variables (x, y, z) rather than all three simultaneously. In our 37-step collision attacks, for example, the differential conditions on $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$ are satisfied in the pre-processing phase of message modification, whereas the conditions appearing after step 16 are verified only in the subsequent verifying phase, which largely determines the overall time complexity. Consequently, it is essential to count these conditions precisely and minimize them. For instance, when counting the number of bit conditions on E_{16} , the input of IF at step 17 is (E_{16}, E_{15}, E_{14}) . In this case, the conditions on (E_{15}, E_{14}) should be ignored, and only those on E_{16} are relevant, representing a *single-variable* scenario. Similarly, at step 18 the input of IF is (E_{17}, E_{16}, E_{15}) , where it suffices to consider only E_{17} and E_{16} , corresponding to the *two-variable* case. However, existing models exhibit two main limitations. Some models [8, 9], such as the *fast filtering model* and the *full model*, ignore the number of bit conditions entirely. In contrast, the CRYPTO 2025 model [10] accounts only for "2-bit conditions" and treats the three inputs (x, y, z) jointly. This latter limitation explains why Li et al. [10] had to perform multiple attempts to determine the minimal number of differential conditions on $(E_i)_{15 \leq i \leq 17}$, as some non-necessary conditions were wrongly included in the objective function. To overcome these issues, our second improvement introduces six *variable-specific condition models* for $w = \text{IF}(x, y, z)$ and $w = \text{MAJ}(x, y, z)$, each designed to handle such scenarios with greater accuracy.

The construction of these new models is described in detail below. It should be emphasized that our tool primarily modifies the modeling of the Boolean functions, while all other components remain the same as in previous methods [8–10], including the models for modular addition, expansion of modular differences, and contradiction detection. By incorporating all relevant bit conditions derived by Boolean functions into the objective function, our model can return the differential trail along with its exact probability. Notice that all previously proposed models require manual reevaluation of the actual probability since they didn't pay too much attention to conditions derived by Boolean functions. These models always ignore all these conditions or take non-necessary ones into account.

To maintain consistency with prior works, we adopt the same representation of signed difference as in [7–10, 12, 25]. Specifically, for a binary variable κ , its signed difference $\nabla \kappa$ is represented as $(\kappa_v, \kappa_d) \in \{(0, 0), (0, 1), (1, 1)\}$, corresponding to $[=]$, $[\mathbf{n}]$ and $[\mathbf{u}]$, respectively. Our implementation is built on the SAT/SMT solver STP.

4.1 Modeling Full Bit Conditions for the IF Function

We refine the original *full model* of $w = \text{IF}(x, y, z)$ [8, 9] by introducing a binary pair (t_1, t_2) to encode the number of implicit conditions in the differential transi-

tions, where $(t_1, t_2) \in \{(0, 0), (0, 1), (1, 1)\}$. Specifically, $(t_1, t_2) = (0, 0)$ indicates no condition, $(0, 1)$ indicates one condition, and $(1, 1)$ indicates two conditions.

For instance, when $(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i]) = [==nn]$, an additional constraint $x[i] = 0$ arises, corresponding to $(t_1, t_2) = (0, 1)$. In this manner, all bit conditions introduced by the IF function are systematically captured. The complete set of refined propagation rules for IF is summarized in Table 4, which we refer to as (x, y, z) -based modeling.

Table 4: Rules for (x, y, z) -based modeling on the function IF

$[===, ***, 00],$
$[==nn, 0**, 01], [==n=, 1**, 01], [==uu, 0**, 01], [==u=, 1**, 01],$
$[=n==, 0**, 01], [=n=n, 1**, 01], [=nuu, 0**, 01], [=nun, 1**, 01], [=nnn, ***, 00],$
$[=u==, 0**, 01], [=u=u, 1**, 01], [=unn, 0**, 01], [=unu, 1**, 01], [=uuu, ***, 00],$
$[n===, * \bullet \bullet, 01], [n==u, *01, 11], [n=n=, *10, 11], [n=n=, *0*, 01], [n=nn, *1*, 01],$
$[n=uu, *0*, 01], [n=u=, *1*, 01], [nn=n, **0, 01], [nn==, **1, 01], [nnnn, ***, 00],$
$[nnu=, ***, 00], [nu==, **0, 01], [nu=u, **1, 01], [nun=, ***, 00], [nuuu, ***, 00],$
$[u===, * \bullet \bullet, 01], [u==n, *01, 11], [u==u, *10, 11], [u=nn, *0*, 01], [u=n=, *1*, 01],$
$[u=u=, *0*, 01], [u=uu, *1*, 01], [un==, **0, 01], [un=n, **1, 01], [unnn, ***, 00],$
$[unu=, ***, 00], [uu=u, **0, 01], [uu==, **1, 01], [uun=, ***, 00], [uuuu, ***, 00].$

$\dagger [\bullet \bullet]$ denotes a "2-bit condition" where the two bits are equal.

We can convert the above 45 transition rules into 45 corresponding tuples of the form

$$(x_v[i], x_d[i], y_v[i], y_d[i], z_v[i], z_d[i], w_v[i], w_d[i], x, y, z, t_1, t_2).$$

Using LogicFriday, we can easily derive the minimized CNF representing the constraints on this tuple.

4.2 Modeling IF and MAJ with Variable-Specific Conditions

To overcome the limitations of joint modeling, we introduce six new models for $w = \text{IF}(x, y, z)$ and $w = \text{MAJ}(x, y, z)$, each designed to minimize the number of bit conditions in a more flexible manner. The key idea is to allow the modeling process to focus on one or two input variables, rather than counting the conditions on all three variables simultaneously. This refinement enables a more precise reduction in the total number of bit conditions in specific scenarios.

To clearly illustrate the applicability of our approach, we categorize variable-specific condition modeling into six distinct scenarios. Each scenario counts the number of bit conditions by restricting the focus to only a subset of the input variables while ignoring the others:

1. x -based modeling: Only the bit conditions on x are counted, while both y and z are ignored;

2. y -based modeling: Only the bit conditions on y are counted, while both x and z are ignored;
3. z -based modeling: Only the bit conditions on z are counted, while both x and y are ignored;
4. (x, y) -based modeling: Only the bit conditions on (x, y) are counted, while z is ignored;
5. (x, z) -based modeling: Only the bit conditions on (x, z) are counted, while y is ignored;
6. (y, z) -based modeling: Only the bit conditions on (y, z) are counted, while x is ignored.

To enhance clarity, we use $w = \text{IF}(x, y, z)$ as an example to illustrate the six modeling scenarios in detail. For $(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i]) = [\text{==nn}]$, the propagation requires the condition $x[i] = 0$. Depending on the chosen modeling strategy, the number of bit conditions that must be counted varies: under x -based, (x, y) -based and (x, z) -based modeling, one bit condition is counted, whereas no bit conditions are counted under y -based, z -based and (y, z) -based modeling. This differs from the rules summarized in Table 4. By performing the same analysis, we can derive the new propagation rules for the Boolean function IF. The same approach can be directly applied to MAJ, and the resulting rules for both functions are presented in Table 5, Table 6 and Table 7, respectively.

By incorporating these new models for Boolean functions into the open-source framework [8], we develop a new automated tool for SHA-2. This tool captures all relevant Boolean function conditions while avoiding the limitation of counting only joint conditions across all three inputs. Consequently, it eliminates the repeated trial procedures reported in [10], where Li et al. had to perform multiple attempts to identify the minimal number of differential conditions on $(E_i)_{15 \leq i \leq 17}$. Overall, our approach enhances both the accuracy and efficiency of searching for differential characteristics.

5 Improved Collision Attacks on SHA-256

Using the local collisions identified in Table 3 and our new search tool, we present the first valid collision attacks on 37-step and 36-step SHA-256, respectively. These results improve upon the best known collision attacks by 6 and 5 steps, respectively. They are also the first ones that cover more than half of the total steps of SHA-256. In the following, we describe the attack procedures in detail.

5.1 The First Collision Attack on 37-Step SHA-256

For 37-step SHA-256, the local collision spans 18 steps (steps 6–23), with nonzero message differences occurring in 6 words ($W_6, W_7, W_9, W_{14}, W_{22}, W_{23}$). Nevertheless, to establish a valid attack, two critical challenges must be addressed beyond identifying a local collision:

1. identifying a suitable differential characteristic in (A_i, E_i, W_i) and ensuring that the probabilities of the uncontrolled parts are as high as possible;

Table 5: Variable-specific condition models for the Boolean function IF in the *single-variable* scenario.

Rules for x -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
[====,***,0], [==nn,0**,1], [==n=,1**,1], [==uu,0**,1], [==u=,1**,1],	
[=n==,0**,1], [=n=n,1**,1], [=nuu,0**,1], [=nun,1**,1], [=nnn,***,0],	
[u==,0**,1], [u=u,1**,1], [=unn,0**,1], [=unu,1**,1], [=uuu,***,0],	
[n===,* ●● ,0], [n==u,* 01 ,0], [n==n,* 10 ,0], [n=n=,* 0* ,0], [n=nn,* 1* ,0],	
[n=uu,* 0* ,0], [n=u=,* 1* ,0], [nn=n,* 0 ,0], [nn==,* 1 ,0], [nnnn,***,0],	
[nnu=,***,0], [nu==,* 0 ,0], [nu=u,* 1 ,0], [nun=,***,0], [nuuu,***,0],	
[u===,* ●● ,0], [u==n,* 01 ,0], [u=u,* 10 ,0], [u=nn,* 0* ,0], [u=n=,* 1* ,0],	
[u=u=,* 0* ,0], [u=uu,* 1* ,0], [un==,* 0 ,0], [un=n,* 1 ,0], [unnn,***,0],	
[unu=,***,0], [uu=u,* 0 ,0], [uu==,* 1 ,0], [uun=,***,0], [uuuu,***,0].	
Rules for y -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
[====,***,0], [==nn,0**,0], [==n=,1**,0], [==uu,0**,0], [==u=,1**,0],	
[=n==,0**,0], [=n=n,1**,0], [=nuu,0**,0], [=nun,1**,0], [=nnn,***,0],	
[u==,0**,0], [u=u,1**,0], [=unn,0**,0], [=unu,1**,0], [=uuu,***,0],	
[n===,* ●● ,1], [n==u,* 01 ,1], [n==n,* 10 ,1], [n=n=,* 0* ,1], [n=nn,* 1* ,1],	
[n=uu,* 0* ,1], [n=u=,* 1* ,1], [nn=n,* 0 ,0], [nn==,* 1 ,0], [nnnn,***,0],	
[nnu=,***,0], [nu==,* 0 ,0], [nu=u,* 1 ,0], [nun=,***,0], [nuuu,***,0],	
[u===,* ●● ,1], [u==n,* 01 ,1], [u=u,* 10 ,1], [u=nn,* 0* ,1], [u=n=,* 1* ,1],	
[u=u=,* 0* ,1], [u=uu,* 1* ,1], [un==,* 0 ,0], [un=n,* 1 ,0], [unnn,***,0],	
[unu=,***,0], [uu=u,* 0 ,0], [uu==,* 1 ,0], [uun=,***,0], [uuuu,***,0].	
Rules for z -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
[====,***,0], [==nn,0**,0], [==n=,1**,0], [==uu,0**,0], [==u=,1**,0],	
[=n==,0**,0], [=n=n,1**,0], [=nuu,0**,0], [=nun,1**,0], [=nnn,***,0],	
[u==,0**,0], [u=u,1**,0], [=unn,0**,0], [=unu,1**,0], [=uuu,***,0],	
[n===,* ●● ,1], [n==u,* 01 ,1], [n==n,* 10 ,1], [n=n=,* 0* ,0], [n=nn,* 1* ,0],	
[n=uu,* 0* ,0], [n=u=,* 1* ,0], [nn=n,* 0 ,1], [nn==,* 1 ,1], [nnnn,***,0],	
[nnu=,***,0], [nu==,* 0 ,1], [nu=u,* 1 ,1], [nun=,***,0], [nuuu,***,0],	
[u===,* ●● ,1], [u==n,* 01 ,1], [u=u,* 10 ,1], [u=nn,* 0* ,0], [u=n=,* 1* ,0],	
[u=u=,* 0* ,0], [u=uu,* 1* ,0], [un==,* 0 ,1], [un=n,* 1 ,1], [unnn,***,0],	
[unu=,***,0], [uu=u,* 0 ,1], [uu==,* 1 ,1], [uun=,***,0], [uuuu,***,0].	

† [●●] denotes a "2-bit condition" where the two bits are equal.

‡ Blue highlights indicate the bit conditions ignored during counting.

Table 6: Variable-specific condition models for the Boolean function IF in the *two-variable* scenario.

Rules for (x, y) -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
[====,***,0], [==nn,0**,1], [==n=,1**,1], [==uu,0**,1], [==u=,1**,1],	
[=n==,0**,1], [=n=n,1**,1], [=nuu,0**,1], [=nun,1**,1], [=nnn,***,0],	
[=u==,0**,1], [=u=u,1**,1], [=unn,0**,1], [=unu,1**,1], [=uuu,***,0],	
[n===,* ●● ,1], [n==u,* 01 ,1], [n==n,* 10 ,1], [n=n=,*0*,1], [n=nn,*1*,1],	
[n=uu,*0*,1], [n=u=,*1*,1], [nn=n,**0,0], [nn==,** 1 ,0], [nnnn,***,0],	
[nnu=,***,0], [nu==,**0,0], [nu=u,** 1 ,0], [nun=,***,0], [nuuu,***,0],	
[u===,* ●● ,1], [u==n,* 01 ,1], [u=u,* 10 ,1], [u=nn,*0*,1], [u=n=,*1*,1],	
[u=u=,*0*,1], [u=uu,*1*,1], [un==,**0,0], [un=n,** 1 ,0], [unnn,***,0],	
[unu=,***,0], [uu=u,** 0 ,0], [uu==,** 1 ,0], [uun=,***,0], [uuuu,***,0].	
Rules for (x, z) -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
[====,***,0], [==nn,0**,1], [==n=,1**,1], [==uu,0**,1], [==u=,1**,1],	
[=n==,0**,1], [=n=n,1**,1], [=nuu,0**,1], [=nun,1**,1], [=nnn,***,0],	
[=u==,0**,1], [=u=u,1**,1], [=unn,0**,1], [=unu,1**,1], [=uuu,***,0],	
[n===,* ●● ,1], [n==u,* 01 ,1], [n==n,* 10 ,1], [n=n=,*0*,0], [n=nn,* 1 *,0],	
[n=uu,*0*,0], [n=u=,* 1 *,0], [nn=n,**0,1], [nn==,**1,1], [nnnn,***,0],	
[nnu=,***,0], [nu==,**0,1], [nu=u,**1,1], [nun=,***,0], [nuuu,***,0],	
[u===,* ●● ,1], [u==n,* 01 ,1], [u=u,* 10 ,1], [u=nn,*0*,0], [u=n=,*1*,0],	
[u=u=,*0*,0], [u=uu,*1*,0], [un==,**0,1], [un=n,**1,1], [unnn,***,0],	
[unu=,***,0], [uu=u,**0,1], [uu==,**1,1], [uun=,***,0], [uuuu,***,0].	
Rules for (y, z) -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t_1, t_2)$	
[====,***,00], [==nn, 0 **,00], [==n=, 1 **,00], [==uu, 0 **,00], [==u=, 1 **,00],	
[=n==, 0 **,00], [=n=n, 1 **,00], [=nuu, 0 **,00], [=nun, 1 **,00], [=nnn,***,00],	
[=u==, 0 **,00], [=u=u, 1 **,00], [=unn, 0 **,00], [=unu, 1 **,00], [=uuu,***,00],	
[n===,* ●● ,01], [n==u,*01,11], [n==n,*10,11], [n=n=,*0*,01], [n=nn,*1*,01],	
[n=uu,*0*,01], [n=u=,*1*,01], [nn=n,**0,01], [nn==,**1,01], [nnnn,***,00],	
[nnu=,***,00], [nu==,**0,01], [nu=u,**1,01], [nun=,***,00], [nuuu,***,00],	
[u===,* ●● ,01], [u==n,*01,11], [u=u,*10,11], [u=nn,*0*,01], [u=n=,*1*,01],	
[u=u=,*0*,01], [u=uu,*1*,01], [un==,**0,01], [un=n,**1,01], [unnn,***,00],	
[unu=,***,00], [uu=u,**0,01], [uu==,**1,01], [uun=,***,00], [uuuu,***,00].	

† [**●●**] denotes a "2-bit condition" where the two bits are equal.

‡ Blue highlights indicate the bit conditions ignored during counting.

Table 7: The models for the Boolean function MAJ with variable-specific conditions.

Rules for x -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
$[====, ***, 0],$ $[==n=, \bullet\bullet*, 1], [==nn, \bullet\blacktriangle*, 1], [==u=, \bullet\bullet*, 1], [==uu, \bullet\blacktriangle*, 1], [==nnn, ***, 0],$ $[=n=, \bullet\bullet*, 1], [=n=n, \bullet\blacktriangle*, 1], [=u=, \bullet\bullet*, 1], [=u=u, \bullet\blacktriangle*, 1], [=nu=, ***, 0],$ $[n==, * \bullet\bullet, 0], [n==n, * \bullet\blacktriangle, 0], [u==, * \bullet\bullet, 0], [u==u, * \bullet\blacktriangle, 0], [un=, ***, 0],$ $[n=nn, ***, 0], [n=u=, ***, 0], [nn=n, ***, 0], [nnnn, ***, 0], [nnun, ***, 0],$ $[nu=, ***, 0], [nunnn, ***, 0], [nuuu, ***, 0], [=uuu, ***, 0], [u=n=, ***, 0],$ $[u=uu, ***, 0], [un==, ***, 0], [unnn, ***, 0], [unuu, ***, 0], [uu=u, ***, 0],$ $[uunu, ***, 0], [uuuu, ***, 0].$	
Rules for y -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
$[====, ***, 0],$ $[==n=, \bullet\bullet*, 1], [==nn, \bullet\blacktriangle*, 1], [==u=, \bullet\bullet*, 1], [==uu, \bullet\blacktriangle*, 1], [==nnn, ***, 0],$ $[=n=, \bullet\bullet*, 0], [=n=n, \bullet\blacktriangle*, 0], [=u=, \bullet\bullet*, 0], [=u=u, \bullet\blacktriangle*, 0], [=nu=, ***, 0],$ $[n==, * \bullet\bullet, 1], [n==n, * \bullet\blacktriangle, 1], [u==, * \bullet\bullet, 1], [u==u, * \bullet\blacktriangle, 1], [un=, ***, 0],$ $[n=nn, ***, 0], [n=u=, ***, 0], [nn=n, ***, 0], [nnnn, ***, 0], [nnun, ***, 0],$ $[nu=, ***, 0], [nunnn, ***, 0], [nuuu, ***, 0], [=uuu, ***, 0], [u=n=, ***, 0],$ $[u=uu, ***, 0], [un==, ***, 0], [unnn, ***, 0], [unuu, ***, 0], [uu=u, ***, 0],$ $[uunu, ***, 0], [uuuu, ***, 0].$	
Rules for z -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
$[====, ***, 0],$ $[==n=, \bullet\bullet*, 0], [==nn, \bullet\blacktriangle*, 0], [==u=, \bullet\bullet*, 0], [==uu, \bullet\blacktriangle*, 0], [==nnn, ***, 0],$ $[=n=, \bullet\bullet*, 1], [=n=n, \bullet\blacktriangle*, 1], [=u=, \bullet\bullet*, 1], [=u=u, \bullet\blacktriangle*, 1], [=nu=, ***, 0],$ $[n==, * \bullet\bullet, 1], [n==n, * \bullet\blacktriangle, 1], [u==, * \bullet\bullet, 1], [u==u, * \bullet\blacktriangle, 1], [un=, ***, 0],$ $[n=nn, ***, 0], [n=u=, ***, 0], [nn=n, ***, 0], [nnnn, ***, 0], [nnun, ***, 0],$ $[nu=, ***, 0], [nunnn, ***, 0], [nuuu, ***, 0], [=uuu, ***, 0], [u=n=, ***, 0],$ $[u=uu, ***, 0], [un==, ***, 0], [unnn, ***, 0], [unuu, ***, 0], [uu=u, ***, 0],$ $[uunu, ***, 0], [uuuu, ***, 0].$	
Rules for (x, y) -based, (x, z) -based and (y, z) -based modeling	
$(\nabla x[i], \nabla y[i], \nabla z[i], \nabla w[i], x, y, z, t)$	
$[====, ***, 0],$ $[==n=, \bullet\bullet*, 1], [==nn, \bullet\blacktriangle*, 1], [==u=, \bullet\bullet*, 1], [==uu, \bullet\blacktriangle*, 1], [==nnn, ***, 0],$ $[=n=, \bullet\bullet*, 1], [=n=n, \bullet\blacktriangle*, 1], [=u=, \bullet\bullet*, 1], [=u=u, \bullet\blacktriangle*, 1], [=nu=, ***, 0],$ $[n==, * \bullet\bullet, 1], [n==n, * \bullet\blacktriangle, 1], [u==, * \bullet\bullet, 1], [u==u, * \bullet\blacktriangle, 1], [un=, ***, 0],$ $[n=nn, ***, 0], [n=u=, ***, 0], [nn=n, ***, 0], [nnnn, ***, 0], [nnun, ***, 0],$ $[nu=, ***, 0], [nunnn, ***, 0], [nuuu, ***, 0], [=uuu, ***, 0], [u=n=, ***, 0],$ $[u=uu, ***, 0], [un==, ***, 0], [unnn, ***, 0], [unuu, ***, 0], [uu=u, ***, 0],$ $[uunu, ***, 0], [uuuu, ***, 0].$	

† $[\bullet\bullet]$ denotes a "2-bit condition" where the two bits are equal.

‡ $[\bullet\blacktriangle]$ denotes a "2-bit condition" where the two bits are different.

§ Blue highlights indicate the bit conditions ignored during counting.

2. finding a conforming message pair that preserves this differential characteristic.

In the remainder of this section, we first demonstrate how our new search tool addresses the first challenge, and then explain how message modification techniques are employed to construct colliding message pairs.

Search for Differential Characteristics. Our procedure for searching the differential characteristic of 37-step SHA-256 is summarized below.

- Step 1: **Minimize the Hamming weight of $(\nabla W_{22}, \nabla W_{23})$.** Specifically, determine the minimal value of $O_1 = \mathbb{H}(\nabla W_{22}) + \mathbb{H}(\nabla W_{23})$, under the constraint that nonzero differences occur only in the 6 expanded message words $(W_6, W_7, W_9, W_{14}, W_{22}, W_{23})$. At this step, the goal is solely to minimize O_1 , without specifying the concrete message differences.
- Step 2: **Minimize the Hamming weight of ∇W_i .** Specifically, under the constraints

$$\begin{aligned} \forall i \in [0, 36] \text{ and } i \notin \{6, 7, 9, 14, 22, 23\} : \delta W_i &= 0, \\ \mathbb{H}(\nabla W_{22}) + \mathbb{H}(\nabla W_{23}) &= O_1, \end{aligned}$$

determine the minimal value of $O_2 = \sum_{i=0}^{36} \mathbb{H}(\nabla W_i)$. Again, the specific message differences are not fixed at this step, and the objective is to obtain the minimal value O_2 .

- Step 3: **Minimize the number of differential conditions on $(E_i)_{16 \leq i \leq 23}$.** The differential conditions consist of two components: the Hamming weight and the bit conditions derived from the Boolean function. For the Boolean function IF in Step 17, with inputs (E_{16}, E_{15}, E_{14}) , an accurate count of the bit conditions on E_{16} requires the use of our x -based modeling. Similarly, for the Boolean function IF in Step 18, the (x, y) -based modeling is employed to evaluate the bit conditions on (E_{17}, E_{16}) . For the subsequent steps, applying the (x, y, z) -based modeling is sufficient. In addition to the constraints at Step 2, we further impose $\sum_{i=0}^{36} \mathbb{H}(\nabla W_i) = O_2$ and determine the minimal value of $O_3 = \sum_{i=16}^{23} (\mathbb{H}(\nabla E_i) + \mathbb{BD}(E_i))$, where $\mathbb{BD}(E_i)$ denotes the bit conditions on E_i . Additionally, the value O_3 must admit a solution for $(\nabla A_i, \nabla E_i, \nabla W_i)$ with $0 \leq i \leq 36$ to allow a 37-step collision attack.
- Step 4: **Minimize the Hamming weight of ∇A_i .** In this step, we further add the condition $\sum_{i=16}^{23} (\mathbb{H}(\nabla E_i) + \mathbb{BD}(E_i)) = O_3$ based on the constraints of Step 3. Under these constraints, we aim to find the minimal value of $O_4 = \sum_{i=0}^{36} \mathbb{H}(\nabla A_i)$ such that there exists a solution of a 37-step collision-generating differential characteristic.
- Step 5: **Minimize the Hamming weight of ∇E_i .** On the basis of Step 4, we impose the condition $\sum_{i=0}^{36} \mathbb{H}(\nabla A_i) = O_4$. The goal is to obtain and output a valid solution for $(\nabla A_i, \nabla E_i, \nabla W_i)_{0 \leq i \leq 36}$ that minimizes $\sum_{i=0}^{36} \mathbb{H}(\nabla E_i)$. Since contradictions frequently occur in SHA-256, we also incorporate a model for value transitions to ensure the validity of the resulting differential characteristic.

Following the above procedure, we successfully identified a 37-step differential characteristic, which is shown in Table 8. Additional conditions that impact the time complexity of the attack are also included in this table. Furthermore, to demonstrate that the new 37-step differential characteristic is valid, we provide a SFS collision instance shown in Table 9.

Finding Confirming Message Pairs. Our procedure for finding confirming message pairs primarily follows the strategy proposed in [9], which adopts a memory-efficient technique that employs two message blocks (M_0, M_1) to convert SFS collisions into collisions. The whole attack procedure consists of three phases: (1) a *pre-processing phase* that constructs a table to store partial solutions for M_1 , (2) a *matching phase* that connects M_0 and M_1 , and (3) a *verifying phase* that checks the remaining uncontrolled conditions. However, as the number of attack steps increases, the degrees of freedom in the message words and the overall complexity become more difficult to manage, requiring more careful analysis. In the following, we provide a detailed description of the procedure for constructing confirming message pairs.

Pre-processing phase. The aim of this phase is to obtain a sufficient number of valid solutions for $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$. To achieve this, we adopt a two-step strategy inspired by Li et al. [9]:

- Step 1: **Generating starting points.** We begin by identifying valid solutions for $(A_1, \dots, A_{15}, E_5, \dots, E_{15}, W_9, \dots, W_{15})$ since the conditions on these states are typically dense. These states are referred to as *starting points*. Specifically, we derive all differential conditions on these states from our 37-step differential characteristic and use the SAT-based model that describes value transitions [11, 15] to cover this part. To ensure sufficient degrees of freedom for the *verifying phase*, we require that $(A_1, \dots, A_4, A_{12}, \dots, A_{15}, E_5, \dots, E_9, E_{12}, \dots, E_{15})$ of each obtained starting point must be distinct. We denote the total number of these starting points as N_{start} .
- Step 2: **Deriving valid A_{-1} from starting points.** For each starting point, we enumerate all possible (W_8, E_4) according to the state update function:

$$E_8 = A_4 \boxplus \textcolor{blue}{E}_4 \boxplus \Sigma_1(E_7) \boxplus \text{IF}(E_7, E_6, E_5) \boxplus K_8 \boxplus \textcolor{red}{W}_8.$$

If there are n_1 and n_2 bit conditions on W_8 and E_4 , respectively, the time complexity of finding valid (W_8, E_4) pairs is $\min(2^{32-n_1}, 2^{32-n_2})$. In other words, we exhaust the variable with more conditions and filter its value with conditions on the other. Thus, the expected number of valid (W_8, E_4) is $2^{32-n_1-n_2}$. For each valid pair (W_8, E_4) , we can uniquely calculate corresponding A_0 :

$$A_4 = E_4 \boxplus \textcolor{blue}{A}_0 \boxplus \Sigma_0(A_3) \boxplus \text{MAJ}(A_3, A_2, A_1).$$

Similarly, for each valid tuple (W_8, E_4, A_0) , we enumerate all possible (W_7, E_3) that satisfy:

$$E_7 = A_3 \boxplus \textcolor{blue}{E}_3 \boxplus \Sigma_1(E_6) \boxplus \text{IF}(E_6, E_5, E_4) \boxplus K_7 \boxplus \textcolor{red}{W}_7.$$

Table 9: The SFS colliding message pair for 37 steps of SHA-256.

<i>CV</i>	0f4252be e2d5d87a 56b40fde 6ab7e678 3dfb0dcf 6ac73c9d 587ec5f5 67bbd7dc
<i>M</i>	5a69a8e4 45ff466e afec8126 2d74afe7 54780c76 94b9dae7 675ce76b 107ffeb9 be7baa67 2653bae8 45b576c8 0de40fc1 2d9ea187 26b93c1b 31f1ac39 24de0094
<i>M'</i>	5a69a8e4 45ff466e afec8126 2d74afe7 54780c76 94b9dae7 873ee7a6 108002b9 be7baa67 28d3bae9 45b576c8 0de40fc1 2d9ea187 26b93c1b 31f1a839 24de0094
hash	d08a6613 7d52c71f e2594bf6 231a4580 76c30bc4 d0bec97d 1c832e47 70298870

The corresponding A_{-1} can then be obtained from:

$$A_3 = E_3 \boxminus \textcolor{red}{A}_{-1} \boxplus \Sigma_0(A_2) \boxplus \text{MAJ}(A_2, A_1, A_0).$$

Suppose there are n_3 and n_4 bit conditions on W_7 and E_3 , respectively, the time complexity of identifying all valid (W_7, E_3, A_{-1}) triples is $\min(2^{32-n_3}, 2^{32-n_4})$, and the expected number of valid solutions is $2^{32-n_3-n_4}$.

Following the above procedure, we can get $N_{\text{pre}} = N_{\text{start}} \cdot 2^{64-n_1-n_2-n_3-n_4}$ valid solutions of $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$. These solutions are stored in a table denoted by **TAB** indexed by A_{-1} . Since we call the solver to actually solve solutions of N_{start} starting points, the time complexity of Step 1 is practical and negligible. The overall time complexity of the **pre-processing phase** is estimated as:

$$T_{\text{pre}} = N_{\text{start}} \cdot (\min(2^{32-n_1}, 2^{32-n_2}) + 2^{32-n_1-n_2} \times \min(2^{32-n_3}, 2^{32-n_4})).$$

The memory complexity equals the size of the **TAB** table:

$$M = N_{\text{pre}} = N_{\text{start}} \cdot 2^{64-n_1-n_2-n_3-n_4}.$$

Matching phase. In this phase, the two message blocks M_0 and M_1 are connected by explicitly matching A_{-1} . Throughout the process, several additional conditions must also be satisfied.

Step 1: **Deriving CV_1 from M_0 .** Randomly select the first message block M_0 and compute the corresponding output

$$CV_1 = (A_{-4}, \dots, \textcolor{red}{A}_{-1}, E_{-4}, \dots, E_{-1}),$$

which serves as the chaining input for the second message block M_1 .

Step 2: **Matching A_{-1} .** Find a match of A_{-1} in **TAB** and for each associated value of $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$, proceed to Step 3. If no match is found, or all corresponding values have been considered, return to Step 1.

Step 3: **Connecting M_0 and M_1 .** Before entering the next phase, the chaining value CV_1 derived from M_0 , together with the above state tuple $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$, has already been determined. The objective now is to ensure a consistent connection between M_0 and M_1 . To achieve this, the remaining states must be computed based on these fixed values. In particular, this requires determining the intermediate values (E_0, E_1, E_2) and the message words $(W_0, W_1, W_2, W_3, W_4, W_5, W_6)$.

We first determine (E_0, E_1, E_2) so that the derived states (A_0, A_1, A_2) remain consistent with those obtained from CV_1 . This requires satisfying the following relations:

$$\begin{aligned} A_0 &= E_0 \boxplus A_{-4} \boxplus \Sigma_0(A_{-1}) \boxplus \text{MAJ}(A_{-1}, A_{-2}, A_{-3}), \\ A_1 &= E_1 \boxplus A_{-3} \boxplus \Sigma_0(A_0) \boxplus \text{MAJ}(A_0, A_{-1}, A_{-2}), \\ A_2 &= E_2 \boxplus A_{-2} \boxplus \Sigma_0(A_1) \boxplus \text{MAJ}(A_1, A_0, A_{-1}). \end{aligned}$$

If all conditions on (E_0, E_1, E_2) hold, we next compute (W_4, W_5, W_6) to ensure that the states (E_4, E_5, E_6) are also consistent with those derived from CV_1 :

$$\begin{aligned} E_4 &= A_0 \boxplus E_0 \boxplus \Sigma_1(E_3) \boxplus \text{IF}(E_3, E_2, E_1) \boxplus K_4 \boxplus W_4, \\ E_5 &= A_1 \boxplus E_1 \boxplus \Sigma_1(E_4) \boxplus \text{IF}(E_4, E_3, E_2) \boxplus K_5 \boxplus W_5, \\ E_6 &= A_2 \boxplus E_2 \boxplus \Sigma_1(E_5) \boxplus \text{IF}(E_5, E_4, E_3) \boxplus K_6 \boxplus W_6. \end{aligned}$$

Finally, if the above conditions are met, we compute (W_0, W_1, W_2, W_3) such that (E_0, E_1, E_2, E_3) also align with those in CV_1 :

$$\begin{aligned} E_3 &= A_{-1} \boxplus E_{-1} \boxplus \Sigma_1(E_2) \boxplus \text{IF}(E_2, E_1, E_0) \boxplus K_3 \boxplus W_3, \\ E_2 &= A_{-2} \boxplus E_{-2} \boxplus \Sigma_1(E_1) \boxplus \text{IF}(E_1, E_0, E_{-1}) \boxplus K_2 \boxplus W_2, \\ E_1 &= A_{-3} \boxplus E_{-3} \boxplus \Sigma_1(E_0) \boxplus \text{IF}(E_0, E_{-1}, E_{-2}) \boxplus K_1 \boxplus W_1, \\ E_0 &= A_{-4} \boxplus E_{-4} \boxplus \Sigma_1(E_{-1}) \boxplus \text{IF}(E_{-1}, E_{-2}, E_{-3}) \boxplus K_0 \boxplus W_0. \end{aligned}$$

In our attacks, no conditions are imposed on (W_0, W_1, W_2, W_3) . We denote by N_{pro} the total number of conditions on $(E_0, E_1, E_2, W_4, W_5, W_6)$ that must be satisfied in this phase. If all these conditions are met, we proceed to the next phase to verify the remaining constraints. Otherwise, we return to Step 2 to search for a new match of A_{-1} and thereby obtain another tuple $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$.

Verifying phase. After completing the previous two phases, all message words $(W_0, \dots, W_{16})$ and state variables $(A_{-4}, \dots, A_{15}, E_{-4}, \dots, E_{15})$ are fully determined. The remaining task is to verify whether the subsequent conditions are satisfied, which involve

$$(A_{16}, \dots, A_{23}, E_{16}, \dots, E_{23}, W_{22}, W_{23}).$$

Denote the number of all these conditions as β . If all these β conditions hold, we obtain a valid colliding pair (M_0, M_1) and (M_0, M'_1) . Otherwise, the procedure returns to the **matching phase** to search for a new match.

Based on the analysis above, when a specific A_{-1} is matched, the probability of successfully generating a colliding message pair is $2^{-N_{\text{pro}}-\beta}$. Additionally, the matching probability of A_{-1} is 2^{-32} . We use N_{test} to denote the number of M_0 that need to be tried, which must satisfy the following equation:

$$N_{\text{test}} \times N_{\text{pre}} \times 2^{-32} \geq \frac{1}{2^{-N_{\text{pro}}-\beta}},$$

where $N_{\text{pre}} = N_{\text{start}} \cdot 2^{64-n_1-n_2-n_3-n_4}$. Consequently, the minimal number of M_0 that needs to be tried to find a colliding pair averagely is

$$N_{\text{test}} = \frac{2^{N_{\text{pro}}+\beta+32}}{N_{\text{pre}}}$$

Accordingly, the time complexity of the final two phases can be estimated as

$$T_{\text{match}} + T_{\text{verify}} = N_{\text{test}} + 2^{N_{\text{pro}}+\beta}.$$

Including the cost of the **pre-processing phase**, the total time complexity is given by $T_{\text{pre}} + T_{\text{match}} + T_{\text{verify}}$ ⁴ and the memory complexity is $M = N_{\text{pre}}$.

Complexity Analysis. According to Table 8 and the above analysis, we have

$$n_1 = 0, n_2 = 12, n_3 = 30, n_4 = 0, N_{\text{pro}} = 27, \beta = 95.$$

For N_{start} , we generated a total of 7,643 $\approx 2^{12.9}$ starting points using the SAT-based tool, which took us about 7 days. Hence, the time complexity of the **pre-processing phase** is

$$\begin{aligned} T_{\text{pre}} &= N_{\text{start}} \cdot (\min(2^{32-n_1}, 2^{32-n_2}) + 2^{32-n_1-n_2} \times \min(2^{32-n_3}, 2^{32-n_4})) \\ &= 2^{12.9} \times (2^{32-12} + 2^{32-12} \times 2^{32-30}) \\ &\approx 2^{35.3}. \end{aligned}$$

Based on these $N_{\text{start}} = 2^{12.9}$ starting points, the expected number of possible states $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$ is

$$N_{\text{pre}} = N_{\text{start}} \cdot 2^{64-n_1-n_2-n_3-n_4} = 2^{12.9} \times 2^{22} = 2^{34.9},$$

and these states are stored in TAB. Consequently, the expected number of M_0 values to be tested on average is

$$N_{\text{test}} = \frac{2^{N_{\text{pro}}+\beta+32}}{N_{\text{pre}}} = \frac{2^{27+95+32}}{2^{34.9}} = 2^{119.1}.$$

⁴ Our estimates rely on the heuristic assumption that the intermediate chaining value after processing M_0 is approximately unbiased. The attack also requires accessing the TAB, whose cost is negligible compared to the dominant complexity.

Therefore, the total time complexity of the 37-step collision attack on SHA-256 is estimated as

$$T = T_{\text{pre}} + T_{\text{match}} + T_{\text{verify}} \approx 2^{35.2} + N_{\text{test}} + 2^{N_{\text{pro}} + \beta} = 2^{35.3} + 2^{119.1} + 2^{122} \approx 2^{122.2},$$

and the memory complexity is determined by the size of TAB, i.e., $N_{\text{pre}} = 2^{34.9}$.

A Further Time Complexity Reduction. As suggested during the review process, a slight modification of the matching phase, i.e., simultaneously matching (A_{-2}, A_{-1}) instead of only A_{-1} , can further reduce the time complexity. In this case, when constructing the table TAB, all possible pairs (W_6, E_2) are enumerated and the values $(A_{-2}, \dots, A_{15}, E_2, \dots, E_{15}, W_6, \dots, W_{15})$ are stored. In the final verifying phase, this eliminates the need to check the 27 bit conditions on W_6 . As a result, the overall time complexity is reduced to $2^{119.1}$ at the cost of increasing the size of TAB to $2^{39.9}$.

5.2 The First Collision Attack on 36-Step SHA-256

In our 36-step collision attack on SHA-256, we employ the local collision $(W_5, W_7, W_8, W_{13}, W_{21}, W_{22})$, which spans 18 steps (5–22). The overall attack procedure closely follows that of the 37-step attack. The only difference during the search for characteristics is the insertion of an additional step between Step 4 and Step 5 to minimize the number of conditions on (E_3, E_4) . This refinement requires the application of both our (y, z) -based modeling and z -based modeling for the Boolean function IF. The resulting new differential characteristic is summarized in Table 10, and we also present a SFS colliding message pair in Table 11 to demonstrate the effectiveness of the derived trail.

Complexity Analysis. Using Table 10 and the preceding analysis, we obtain

$$n_1 = 18, n_2 = 8, n_3 = 23, n_4 = 12, N_{\text{pro}} = 22, \beta = 53.$$

Since (W_7, E_3) involves a total of 35 conditions, we found that many candidate starting points $(A_1, \dots, A_{15}, E_5, \dots, E_{15}, W_9, \dots, W_{15})$ identified in Step 1 of the *pre-processing phase* always do not admit any valid solution for (W_7, E_3) . To address this issue, we merge the two substeps of the *pre-processing phase* and employ a SAT-based model that explicitly captures the value transitions over this part, i.e., we directly solve for $(A_{-1}, \dots, A_{15}, E_3, \dots, E_{15}, W_7, \dots, W_{15})$ with a solver during the *pre-processing phase*. Ultimately, we obtain a total of 6,516 $\approx 2^{12.6}$ valid solutions within two weeks. These solutions can be stored in TAB with size $N_{\text{pre}} = 2^{12.6}$. The expected number of M_0 is

$$N_{\text{test}} = \frac{2^{N_{\text{pro}} + \beta + 32}}{N_{\text{pre}}} = \frac{2^{22 + 53 + 32}}{2^{12.6}} = 2^{94.4}.$$

Notice that the *pre-processing phase* is performed only once, and these solutions are obtained practically. Hence, the time complexity of this phase is negligible compared to the overall attack. Thus, we approximate its time complexity

T_{pre} as 0. Consequently, the overall time complexity is given by

$$T = T_{\text{pre}} + T_{\text{match}} + T_{\text{verify}} \approx 0 + N_{\text{test}} + 2^{N_{\text{pro}}+\beta} = 0 + 2^{94.4} + 2^{75} \approx 2^{94.4},$$

and the memory complexity is $N_{\text{pre}} = 2^{12.6}$.

Discussion. At EUROCRYPT 2024, Li et al. [8] observed that, due to the structural similarity between SHA-256 and SHA-512, a collision attack on r steps of SHA-256 should, in principle, also be applicable to r steps of SHA-512. Following this reasoning, we expect that the local collisions identified in attacking 36/37-step SHA-256 could also improve the best-known collision attacks on SHA-512. However, we have not yet discovered any effective differential characteristics for SHA-512. Typically, the attack stalls in the final two steps of the differential characteristic search. We believe that the challenges arise from two main factors. First, the larger state size of SHA-512 significantly complicates the corresponding models, making them harder to solve. Second, the early steps of the search impose tight constraints, which limit the search space and make it harder to identify valid characteristics. Therefore, if more efficient models could be developed to enhance the search efficiency, it is likely that better results could be achieved. Additionally, relaxing the constraints in the early steps of the differential characteristic search could broaden the search space and improve the overall efficiency of the attack, enabling the identification of better differential characteristics. We leave these challenges as directions for future research, since overcoming them would not only improve the efficiency of differential characteristic searches but also provide deeper insights into the security margins of SHA-512.

6 Conclusion

This paper presents significant advancements in collision attacks on SHA-256. We first developed an automated tool for identifying high-quality local collisions, which was crucial for extending the attack to more steps. With this tool, we successfully discovered local collisions that enable the first 37-step collision attack on SHA-256. Second, a more holistic model searching for signed differential trails is given, which overcomes the incompleteness of objective functions by systematically counting bit conditions derived from Boolean functions. This new model enables more precise control over previously uncontrolled conditions and improves the accuracy of probability estimation. Benefiting from these two contributions, we achieved the first 37-step collision attack on SHA-256, surpassing the previous best attack by 6 steps. Notice that this is the first improvement in the number of attacked steps in over twelve years. Remarkably, this also marks a successful attack on more than half of the full-round SHA-256, more than twenty years after its initial publication.

Acknowledgments. We sincerely thank the anonymous reviewers for providing valuable comments to help us improve the overall quality of the paper.

Table 10: The differential characteristic for 36-step SHA-256.

i	∇A_i	∇E_i	∇W_i
-4	=====	=====	
-3	=====	=====	
-2	=====	=====	
-1	=====	=====	
0	=====	=====	=====
1	=====	=====	=====
2	=====	=====	=====
3	=====	=====0=====0=====	=====
4	=====	=====0=====1=====0=====0=====	=====
5	=====n=====u=====	0111nnuuuuuu00001unn11011101011	nnuuuuuuuuuu=====unnn=====
6	=====u=====	01u=0111110n000u10=10n1n1u1n11=1	=====
7	=====	111=nnuu100un111u=01011nnuu=u1==	n=u=u=====u=n=====un=u=====u=n
8	=====	=1100000=010101=0=====011011=0=====	=====nn=====u0n=====0u=====
9	=====n=====	001100111nu1u101011=u0001nu0110=	=====00n=====
10	=====u=====u=====n=====	1n1u=u1010110uu00=1010=0u010=110	=====
11	n=====u=====u=====	unu10n0u=1001101u=n110=0011n1110	=====
12	=====	1=0u01un10001n0=1n1nn0n000u10=n1	=====
13	=====un=====nu=====	1101u11=111n010111nu=11=110n=10	=====unnnnnuuuuuuuuuu=====
14	=====u=====n=====	1101nu0011unnu10001uu=1u111uu01n	=====
15	=====	=====n0=====0u1n0=====000u=11=10u=11	=====
16	=====	=====0=====001=====10=====1=====1	=====
17	=====	=====1=====1=u=====n=====0=====	=====
18	=====	=====1=====u=====1=====n=====	=====
19	=====	=====1=====0=====1=====0=====	=====
20	=====	=====1=====1=====	=====
21	=====	=====	=====n=====1=u=1=====
22	=====	=====	=====1=n=0=====u=====
23	=====	=====	=====
24	=====	=====	=====
25	=====	=====	=====
26	=====	=====	=====
27	=====	=====	=====
28	=====	=====	=====
29	=====	=====	=====
30	=====	=====	=====
31	=====	=====	=====
32	=====	=====	=====
33	=====	=====	=====
34	=====	=====	=====
35	=====	=====	=====
A_{16}	$A_{16}[1] = A_{15}[1], A_{16}[14] = A_{15}[14].$		
E_{17}	$E_{17}[1] = E_{16}[1], E_{17}[14] = E_{16}[14],$ $E_{17}[0] = E_{17}[5], E_{17}[24] = E_{17}[29], E_{17}[24] \neq E_{17}[6], E_{17}[14] \neq E_{17}[1], E_{17}[16] = E_{17}[30], E_{17}[6] \neq E_{17}[25].$		
E_{18}	$E_{18}[6] \neq E_{18}[20], E_{18}[28] = E_{18}[15], E_{18}[27] = E_{18}[0], E_{18}[9] \neq E_{18}[28].$		
W_5	$W_5[1] \neq W_5[18], W_5[0] = W_5[17], W_5[10] = W_5[6], W_5[9] \neq W_5[5], W_5[8] \neq W_5[4].$		
W_{21}	$W_{21}[26] = W_{21}[28], W_{21}[21] = W_{21}[12], W_{21}[18] \neq W_{21}[20], W_{21}[17] \neq W_{21}[10].$		
W_{22}	$W_{22}[3] \neq W_{22}[26], W_{21}[31] = W_{22}[24], W_{22}[21] = W_{22}[23].$		
E_3	$E_3[12 \sim 13, 19 \sim 20, 22 \sim 25, 27] = E_4[12 \sim 13, 19 \sim 20, 22 \sim 25, 27].$		
W_7	$W_7[4] \neq W_7[21], W_7[24] = W_7[3], W_7[23] = W_7[19], W_7[22] \neq W_7[1], W_7[21] \neq W_7[17], W_7[24] = W_7[9],$ $W_7[12] \neq W_7[23], W_7[11] \neq W_7[22], W_7[21] \neq W_7[6], W_7[9] \neq W_7[5], W_7[19] \neq W_7[4].$		
W_8	$W_8[3] \neq W_8[31], W_8[0] \neq W_8[11], W_8[31] \neq W_8[10], W_8[7] = W_8[24], W_8[6] \neq W_8[23], W_8[25] = W_8[21],$ $W_8[20] \neq W_8[31], W_8[18] = W_8[29], W_8[17] \neq W_8[13], W_8[25] \neq W_8[10], W_8[8] = W_8[19].$		

Table 11: The SFS colliding message pair for 36 steps of SHA-256.

CV	e66d2bd8 e3edba21 ba127056 6fee2e22 fa0bde22 579c8cad 28035798 cc37783b
M	723e142c 707ad9d2 1dab79ad 44273be9 ed39e9ee 7ff9c7c2 40d1026e 1bf9c78c 82230057 311511c4 1bc6e51b ded9f32d 956ed153 4b561ffe 2d929a18 dde8a8ab
M'	723e142c 707ad9d2 1dab79ad 44273be9 ed39e9ee 8001bfc2 40d1026e 89eda209 9a224047 311511c4 1bc6e51b ded9f32d 956ed153 4b56e000 2d929a18 dde8a8ab
hash	e43637a7 a7ed5162 9a328c44 d9e5591e 3a71407d fae30a7e 66c15de7 9daa5143

This work is supported by the National Key R&D Program of China (Grant No. 2024YFA1013000), the National Natural Science Foundation of China (U2336207, Grant No. 62032014), the National Cryptologic Science Fund of China (2025NCSF01013) and the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM114). Muzhou Li was also supported by the National Natural Science Foundation of China (Grant No. 62502277), the Natural Science Foundation of Shandong Province, China (Grant No. ZR2024QF218).

Generative AI tools (OpenAI's ChatGPT) were used to assist with spelling and grammatical corrections. The authors remain fully responsible for the accuracy and integrity of the final manuscript.

References

1. Biham, E., Chen, R., Joux, A., Carribault, P., Lemuet, C., Jalby, W.: Collisions of SHA-0 and reduced SHA-1. In: Cramer, R. (ed.) *Advances in Cryptology - EUROCRYPT 2005*, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3494, pp. 36–57. Springer (2005), https://doi.org/10.1007/11426639_3
2. Eichlseder, M., Mendel, F., Schl  ffer, M.: Branching heuristics in differential collision search with applications to SHA-512. In: Cid, C., Rechberger, C. (eds.) *Fast Software Encryption - 21st International Workshop, FSE 2014*, London, UK, March 3-5, 2014. Revised Selected Papers. Lecture Notes in Computer Science, vol. 8540, pp. 473–488. Springer (2014), https://doi.org/10.1007/978-3-662-46706-0_24
3. Indestege, S., Mendel, F., Preneel, B., Rechberger, C.: Collisions and other non-random properties for step-reduced SHA-256. In: Avanzi, R.M., Keliher, L., Sica, F. (eds.) *Selected Areas in Cryptography*, 15th International Workshop, SAC 2008, Sackville, New Brunswick, Canada, August 14-15, Revised Selected Papers. Lecture Notes in Computer Science, vol. 5381, pp. 276–293. Springer (2008), https://doi.org/10.1007/978-3-642-04159-4_18
4. Landelle, F., Peyrin, T.: Cryptanalysis of full RIPEMD-128. In: Johansson, T., Nguyen, P.Q. (eds.) *Advances in Cryptology - EUROCRYPT 2013*, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7881, pp. 228–244. Springer (2013), https://doi.org/10.1007/978-3-642-38348-9_14
5. Leurent, G., Peyrin, T.: From collisions to chosen-prefix collisions application to full SHA-1. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part III. Lecture Notes in Computer Science, vol. 11478, pp. 527–555. Springer (2019), https://doi.org/10.1007/978-3-030-17659-4_18
6. Leurent, G., Peyrin, T.: SHA-1 is a shambles: First chosen-prefix collision on SHA-1 and application to the PGP web of trust. In: Capkun, S., Roesner, F. (eds.) *29th USENIX Security Symposium*, USENIX Security 2020, August 12-14, 2020. pp. 1839–1856. USENIX Association (2020), <https://www.usenix.org/conference/usenixsecurity20/presentation/leurent>

7. Li, Y., Liu, F., Wang, G.: Automating collision attacks on RIPEMD-160. *IACR Trans. Symmetric Cryptol.* **2023**(4), 112–142 (2023), <https://doi.org/10.46586/tosc.v2023.i4.112-142>
8. Li, Y., Liu, F., Wang, G.: New records in collision attacks on SHA-2. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zurich, Switzerland, May 26–30, 2024, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 14651, pp. 158–186. Springer (2024), https://doi.org/10.1007/978-3-031-58716-0_6
9. Li, Y., Liu, F., Wang, G., Dong, X., Sun, S.: The first practical collision for 31-step SHA-256. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security*, Kolkata, India, December 9–13, 2024, Proceedings, Part VII. *Lecture Notes in Computer Science*, vol. 15490, pp. 237–266. Springer (2024), https://doi.org/10.1007/978-981-96-0941-3_8
10. Li, Y., Liu, F., Wang, G., Qian, H., Jia, K., Kong, X.: New collision attacks on round-reduced SHA-512. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part V. *Lecture Notes in Computer Science*, vol. 16004, pp. 200–229. Springer (2025), https://doi.org/10.1007/978-3-032-01901-1_7
11. Liu, F., Isobe, T., Meier, W.: Automatic verification of differential characteristics: Application to reduced gimli. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference*, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 12172, pp. 219–248. Springer (2020), https://doi.org/10.1007/978-3-030-56877-1_8
12. Liu, F., Wang, G., Sarkar, S., Anand, R., Meier, W., Li, Y., Isobe, T.: Analysis of RIPEMD-160: new collision attacks and finding characteristics with MILP. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Lyon, France, April 23–27, 2023, Proceedings, Part IV. *Lecture Notes in Computer Science*, vol. 14007, pp. 189–219. Springer (2023). https://doi.org/10.1007/978-3-031-30634-1_7
13. Mendel, F., Nad, T., Schl  ffer, M.: Finding SHA-2 characteristics: Searching through a minefield of contradictions. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security*, Seoul, South Korea, December 4–8, 2011. Proceedings. *Lecture Notes in Computer Science*, vol. 7073, pp. 288–307. Springer (2011), https://doi.org/10.1007/978-3-642-25385-0_16
14. Mendel, F., Nad, T., Schl  ffer, M.: Improving local collisions: New attacks on reduced SHA-256. In: Johansson, T., Nguyen, P.Q. (eds.) *Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Athens, Greece, May 26–30, 2013. Proceedings. *Lecture Notes in Computer Science*, vol. 7881, pp. 262–278. Springer (2013), https://doi.org/10.1007/978-3-642-38348-9_16
15. Mironov, I., Zhang, L.: Applications of SAT solvers to cryptanalysis of hash functions. In: Biere, A., Gomes, C.P. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2006, 9th International Conference*, Seattle, WA, USA, August 12–

- 15, 2006, Proceedings. Lecture Notes in Computer Science, vol. 4121, pp. 102–115. Springer (2006), https://doi.org/10.1007/11814948_13
16. National Institute of Standards and Technology: FIPS 180-4: Secure Hash Standard (SHS). Federal Information Processing Standards Publication 180-4, U.S. Department of Commerce (2012). <https://doi.org/10.6028/NIST.FIPS.180-4>
17. Sanadhya, S.K., Sarkar, P.: New local collisions for the SHA-2 hash family. In: Nam, K., Rhee, G. (eds.) Information Security and Cryptology - ICISC 2007, 10th International Conference, Seoul, Korea, November 29-30, 2007, Proceedings. Lecture Notes in Computer Science, vol. 4817, pp. 193–205. Springer (2007), https://doi.org/10.1007/978-3-540-76788-6_16
18. Sanadhya, S.K., Sarkar, P.: New collision attacks against up to 24-step SHA-2. In: Chowdhury, D.R., Rijmen, V., Das, A. (eds.) Progress in Cryptology - INDOCRYPT 2008, 9th International Conference on Cryptology in India, Kharagpur, India, December 14-17, 2008. Proceedings. Lecture Notes in Computer Science, vol. 5365, pp. 91–103. Springer (2008), https://doi.org/10.1007/978-3-540-89754-5_8
19. Sanadhya, S.K., Sarkar, P.: Non-linear reduced round attacks against SHA-2 hash family. In: Mu, Y., Susilo, W., Seberry, J. (eds.) Information Security and Privacy, 13th Australasian Conference, ACISP 2008, Wollongong, Australia, July 7-9, 2008, Proceedings. Lecture Notes in Computer Science, vol. 5107, pp. 254–266. Springer (2008), https://doi.org/10.1007/978-3-540-70500-0_19
20. Stevens, M., Bursztein, E., Karpman, P., Albertini, A., Markov, Y.: The first collision for full SHA-1. In: Katz, J., Shacham, H. (eds.) Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part I. Lecture Notes in Computer Science, vol. 10401, pp. 570–596. Springer (2017), https://doi.org/10.1007/978-3-319-63688-7_19
21. Wang, X., Lai, X., Feng, D., Chen, H., Yu, X.: Cryptanalysis of the hash functions MD4 and RIPEMD. In: Cramer, R. (ed.) Advances in Cryptology - EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3494, pp. 1–18. Springer (2005), https://doi.org/10.1007/11426639_1
22. Wang, X., Yin, Y.L., Yu, H.: Finding collisions in the full SHA-1. In: Shoup, V. (ed.) Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3621, pp. 17–36. Springer (2005), https://doi.org/10.1007/11535218_2
23. Wang, X., Yu, H.: How to break MD5 and other hash functions. In: Cramer, R. (ed.) Advances in Cryptology - EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3494, pp. 19–35. Springer (2005), https://doi.org/10.1007/11426639_2
24. Wang, X., Yu, H., Yin, Y.L.: Efficient collision search attacks on SHA-0. In: Shoup, V. (ed.) Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3621, pp. 1–16. Springer (2005), https://doi.org/10.1007/11535218_1
25. Zhang, Z., Li, M., Wang, H., Hou, S., Wang, W., Wang, M.: Improved semi-free-start collision attacks on RIPEMD-160. In: Hanaoka, G., Yang, B. (eds.) Advances

in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part I. Lecture Notes in Computer Science, vol. 16245, pp. 221–252. Springer (2025), https://doi.org/10.1007/978-981-95-5018-0_8